Pseudocodice

Indice

Introduzione allo pseudocodice

Variabili e assegnamenti

Operazioni di base

Costrutti di base

Array

Funzioni

Insiemi

Convenzioni di stile

Introduzione (1/3)

- Nel capitolo precedente abbiamo visto come sono rappresentati i dati di un algoritmo, i numeri
- In questo capitolo fissiamo il linguaggio usato per descrivere gli algoritmi stessi: lo **pseudocodice**
- Lo pseudocodice è abbastanza preciso da descrivere senza ambiguità ogni passo di un algoritmo, e abbastanza semplice da poter essere letto senza conoscere alcun linguaggio di programmazione

Introduzione (2/3)

Gli elementi dello pseudocodice, uno alla volta, dal più semplice al più strutturato:

- variabili, dichiarazioni e assegnamenti, e le operazioni di base su di essi
- i costrutti di base che controllano l'ordine di esecuzione: istruzioni condizionali, cicli, e le istruzioni di controllo dei cicli **break** e **continue**
- array con uno, due o più indici
- funzioni, che danno un nome a un algoritmo, con il suo input e il suo output
- insiemi

Introduzione (3/3)

Ogni costrutto è seguito da esempi,
eseguiti passo per passo su piccole istanze.

Eseguire un algoritmo a mano, tenendo traccia
dei valori delle sue variabili, è il modo migliore
per capire che cosa fa.

Introduzione allo pseudocodice

Introduzione: Introduzione allo pseudocodice

- In questa sezione definiamo che cos'è un **algoritmo** e che cos'è lo **pseudocodice**
- Lo pseudocodice è il linguaggio in cui sono scritti tutti gli algoritmi del corso
- Ne vediamo i vantaggi, i principi fondamentali e le buone pratiche per scriverlo

Algoritmo e pseudocodice

Definizione 1: algoritmo

*Un **algoritmo** è una sequenza finita di istruzioni non ambigue che, a partire da alcuni dati forniti come **input**, produce in un numero finito di passi un risultato fornito come **output**.*

Definizione 2: pseudocodice

*Lo **pseudocodice** è una descrizione di alto livello di un algoritmo che usa un misto di linguaggio naturale e di strutture simili a quelle dei linguaggi di programmazione.*

A che cosa serve lo pseudocodice

- Lo pseudocodice non è fatto per essere eseguito da un calcolatore, ma per comunicare agli esseri umani la logica di un algoritmo
- Lo pseudocodice permette di concentrarsi sulla logica e sulla struttura di un algoritmo senza preoccuparsi della sintassi di uno specifico linguaggio di programmazione
- **Convenzione:** tutte le parole riservate (parole chiave) dello pseudocodice sono scritte in grassetto, ad esempio **if**, **while**, **for**, **return**, ecc.

Vantaggi dello pseudocodice

- Indipendente dal linguaggio: può essere compreso da programmatori che usano linguaggi diversi
- Facile da scrivere e da modificare
- Si concentra sulla logica dell'algoritmo anziché sui dettagli di implementazione
- Facilita la comunicazione tra sviluppatori e persone non tecniche
- Utile per la pianificazione prima della scrittura effettiva del codice

Principi fondamentali

- Usare nomi di variabili chiari e descrittivi
- Mantenere le istruzioni semplici e concise
- Usare l'indentazione per mostrare la struttura e l'annidamento
- Usare costrutti standard (**if-then-else**, **while**, **for**, ecc.)
- Inserire commenti quando necessario per chiarire le operazioni complesse

Buone pratiche

- **Indentazione:** usare un'indentazione coerente per mostrare la struttura a blocchi
- **Variabili:** usare nomi descrittivi in corsivo o in testo normale
- **Commenti:** aggiungere commenti (in linguaggio naturale) per spiegare i passi complessi
- **Input/Output:** specificare chiaramente i parametri di input e l'output (valori restituiti)
- **Numeri di riga:** utili per fare riferimento a righe specifiche
- **Coerenza:** mantenere la stessa notazione e lo stesso stile in tutto il testo

Riepilogo

- Un **algoritmo** è una sequenza finita di istruzioni non ambigue che porta da un input a un output in un numero finito di passi
- Lo **pseudocodice** descrive un algoritmo per gli esseri umani, mescolando linguaggio naturale e strutture simili a quelle dei linguaggi di programmazione; le parole chiave sono in grassetto
- Un buon pseudocodice è chiaro, conciso, indentato e commentato in modo coerente, con input e output specificati chiaramente

Variabili e assegnamenti

Introduzione: Variabili e assegnamenti

- I dati di un algoritmo sono memorizzati in **variabili**: contenitori dotati di un nome, il cui valore può cambiare durante l'esecuzione
- Una variabile viene prima creata con una **dichiarazione** e poi modificata con degli **assegnamenti**
- Dichiarazioni e assegnamenti sono le prime istruzioni di ogni algoritmo

Variabili

- Le variabili devono essere dichiarate prima dell'uso
- L'operatore di dichiarazione è ":="
- La dichiarazione specifica il nome della variabile e il suo valore iniziale

```
1 variabile := valore_iniziale
```

Esempio: dichiarazione di variabili

Sette variabili dichiarate con i loro valori iniziali: sono ammessi valori interi, razionali, $-\infty$ e logici (**true**/**false**).

```
1 n := 5
2 m := 10
3 x := −3/5
4 sum := 0
5 count := 1
6 max := −∞
7 status := true
```

Assegnamento

- L'operatore di assegnamento è "$\leftarrow$"
- Assegna il valore di un'espressione a una variabile dichiarata in precedenza: prima si valuta l'espressione a destra, poi il suo valore sostituisce il vecchio valore della variabile

```
1 variabile ← espressione
```

Esempio: assegnamento

Con le variabili dell'esempio precedente:

1 $n \leftarrow 10$
2 $sum \leftarrow n + m$
3 $count \leftarrow count + 1$

- Dopo i tre assegnamenti si ha $n = 10$, $sum = 10 + 10 = 20$ e $count = 1 + 1 = 2$
- Nell'ultimo assegnamento la stessa variabile compare in entrambi i membri: il vecchio valore 1 viene usato per calcolare il nuovo valore 2

Esempio: scambio dei valori di due variabili

Per scambiare i valori di due variabili x e y serve una terza variabile temporanea *tmp*: il solo assegnamento $x \leftarrow y$ sovrascriverebbe il vecchio valore di x.

```
1 tmp := x
2 x ← y
3 y ← tmp
```

Esecuzione passo per passo con $x = 3$ e $y = 8$:

istruzione	x	y	*tmp*
–	3	8	–
$tmp := x$	3	8	3
$x \leftarrow y$	8	8	3
$y \leftarrow tmp$	8	3	3

Riepilogo

- Una variabile si **dichiara** con ":=", che ne dà il nome e il valore iniziale
- Una variabile si modifica con un **assegnamento** "$\leftarrow$": prima si valuta l'espressione a destra, poi il suo valore sostituisce quello vecchio
- Per scambiare i valori di due variabili serve una variabile temporanea

Operazioni di base

Introduzione: Operazioni di base

- Le espressioni combinano variabili e valori con un piccolo insieme di operazioni
- Elenchiamo le operazioni matematiche, di confronto e logiche, e le istruzioni di input/output
- Compaiono a destra degli assegnamenti e nelle condizioni dei costrutti che seguono

Operazioni matematiche

- $+, -, \times, /$: aritmetica di base
- a mod b: modulo, cioè il resto della divisione intera di a per b
- $\lfloor x \rfloor$: parte intera inferiore (il più grande intero $\leq x$)
- $\lceil x \rceil$: parte intera superiore (il più piccolo intero $\geq x$)
- $|x|$: valore assoluto
- $\max(a, b)$, $\min(a, b)$: massimo e minimo

Operatori di confronto, logici e di input/output

Operatori di confronto:

- $=, \neq$: uguale, diverso
- $<, \leq, >, \geq$: minore, minore o uguale, maggiore, maggiore o uguale

Operatori logici:

- **and**, **or**, **not**: operazioni logiche

Input/Output:

- **read**(variabile): legge un input dall'utente
- **print**(espressione): stampa un valore

Esempio: valutazione di espressioni

- $17 \bmod 5 = 2$, poiché $17 = 3 \cdot 5 + 2$; in particolare, un intero i è multiplo di 3 se e solo se $i \bmod 3 = 0$
- $\lfloor 7/2 \rfloor = \lfloor 3.5 \rfloor = 3$ e $\lceil 7/2 \rceil = \lceil 3.5 \rceil = 4$
- $|-3| = 3$, $\max(-3, 2) = 2$ e $\min(-3, 2) = -3$
- $(5 > 3)$ **and** $(2 > 4)$ è **false**, mentre $(5 > 3)$ **or** $(2 > 4)$ è **true**

Riepilogo

- Operazioni matematiche: aritmetica, mod, parte intera inferiore $\lfloor \cdot \rfloor$ e superiore $\lceil \cdot \rceil$, valore assoluto, max e min
- Gli operatori di confronto ($=, \neq, <, \leq, >, \geq$) e gli operatori logici (**and**, **or**, **not**) danno **true** o **false**
- **read** e **print** scambiano dati con l'utente

Costrutti di base

Introduzione: Costrutti di base

- Le istruzioni dello pseudocodice vengono eseguite una dopo l'altra, dall'alto verso il basso
- Due tipi di costrutti cambiano quest'ordine:
  - le **istruzioni condizionali**, che eseguono un blocco solo quando vale una condizione
  - i **cicli**, che eseguono un blocco più volte
- Con questi costrutti, e con **break** e **continue**, possiamo scrivere ogni algoritmo del corso

Istruzioni condizionali

Cominciamo dalle istruzioni condizionali:
if-then e **if-then-else**.

If-then

- Esegue un blocco di istruzioni solo se una condizione è vera

```
1 if condizione then
2 |   istruzioni
3 end
```

Esempio: if-then (1/2)

Verifica se un numero x è positivo:

```
1 positive := false
2 if x > 0 then
3 |   positive ← true
4 |   print(x è positivo)
5 end
```

Esempio: if-then (2/2)

Esecuzione passo per passo:

Istanza 1: $x = 5$

x	$x > 0$?	*positive*
–	–	false
5	Sì	true

Output: `5 è positivo`

Istanza 2: $x = -3$

x	$x > 0$?	*positive*
–	–	false
-3	No	false

Output: non viene stampato nulla.

If-then-else

- Esegue un blocco se la condizione è vera, e un altro blocco se è falsa

```
1 if condizione then
2 |   istruzioni se vera
3 else
4 |   istruzioni se falsa
5 end
```

Esempio: if-then-else (1/2)

Calcolo del valore assoluto di un numero x:

```
1 result := 0
2 if x ≥ 0 then
3 |   result ← x
4 else
5 |   result ← −x
6 end
```

Esempio: if-then-else (2/2)

Esecuzione passo per passo:

Istanza 1: $x = 7$

x	$x \geq 0$?	*result*
–	–	0
7	Sì	7

Risultato: $result = 7$

Istanza 2: $x = -4$

x	$x \geq 0$?	*result*
–	–	0
-4	No	4

Risultato: $result = 4$

Cicli

Passiamo ora ai cicli,
che eseguono un blocco più volte:
il ciclo **while** e il ciclo **for**.

Ciclo while

- Ripete un blocco di istruzioni finché una condizione è vera
- La condizione viene controllata prima di ogni iterazione
- Se la condizione è inizialmente falsa, il corpo del ciclo non viene mai eseguito

```
1 while condizione do
2 |   istruzioni
3 end
```

Esempio: ciclo while (1/2)

Calcolo della somma dei numeri da 1 a n:

```
1 sum := 0
2 i := 1
3 while i ≤ n do
4 |   sum ← sum + i
5 |   i ← i + 1
6 end
```

Esempio: ciclo while (2/2)

Esecuzione passo per passo. Istanza: $n = 5$.

Ogni riga mostra il valore di i quando la condizione viene controllata, l'esito del controllo, e i valori di *sum* e i alla fine dell'iterazione.

i	$i \leq n$?	*sum*	i (dopo)
–	–	0	1
1	Sì	1	2
2	Sì	3	3
3	Sì	6	4
4	Sì	10	5
5	Sì	15	6
6	No	15	–

Risultato: $sum = 15$

Quando il numero di iterazioni non è noto

Nell'esempio precedente il numero di iterazioni è noto in anticipo.

Il ciclo while è necessario quando non lo è, come nell'esempio seguente.

Esempio: ciclo while (1/3)

Calcolo di quante volte un numero x va dimezzato prima che diventi minore o uguale a una soglia $\varepsilon > 0$:

```
1 count := 0
2 while x > ε do
3 |   x ← x/2
4 |   count ← count + 1
5 end
```

Esempio: ciclo while (2/3)

Esecuzione passo per passo. Istanza 1: $x = 10$, $\varepsilon = 1$

x	*count*	$x > \varepsilon$?	x (dopo)	*count* (dopo)
10	0	Sì	5	1
5	1	Sì	2.5	2
2.5	2	Sì	1.25	3
1.25	3	Sì	0.625	4
0.625	4	No $\Rightarrow$ uscita	–	4

Risultato: $count = 4$

Esempio: ciclo while (3/3)

Esecuzione passo per passo. Istanza 2: $x = 100$, $\varepsilon = 10$

x	*count*	$x > \varepsilon$?	x (dopo)	*count* (dopo)
100	0	Sì	50	1
50	1	Sì	25	2
25	2	Sì	12.5	3
12.5	3	Sì	6.25	4
6.25	4	No $\Rightarrow$ uscita	–	4

Risultato: $count = 4$

Ciclo for

- Ripete un blocco di istruzioni per un numero di volte specificato
- Di solito si usa quando il numero di iterazioni è noto in anticipo

```
1 for variabile := valore_iniziale to valore_finale do
2 |   istruzioni
3 end
```

Il ciclo for come ciclo while

Il ciclo for è un modo compatto di scrivere un particolare ciclo while: la variabile del ciclo viene dichiarata con il valore iniziale, la condizione la confronta con il valore finale, e viene incrementata di 1 alla fine di ogni iterazione.

Il ciclo a sinistra è equivalente al ciclo a destra:

```
1 for i := s to e do
2 |   istruzioni
3 end
```

```
1 i := s
2 while i ≤ e do
3 |   istruzioni
4 |   i ← i + 1
5 end
```

In particolare, il corpo viene eseguito $e - s + 1$ volte se $s \leq e$, e non viene mai eseguito se $s > e$.

Esempio: ciclo for

Calcolo della somma dei numeri da 1 a n con un ciclo for (confrontare con il primo esempio del ciclo while):

```
1 sum := 0
2 for i := 1 to n do
3 |   sum ← sum + i
4 end
```

Istanza: $n = 5$

i	$i \leq n$?	sum
–	–	0
1	Sì	1
2	Sì	3
3	Sì	6
4	Sì	10
5	Sì	15
6	No	15

Risultato: $sum = 15$

Ciclo for con passo

Il ciclo for può usare facoltativamente
un valore di **passo** (*step*), per incrementare
la variabile del ciclo di valori diversi da 1.

Esempio: ciclo for con passo

Calcolo della somma dei primi n numeri pari $(2, 4, \ldots, 2n)$:

```
1 sum := 0
2 for i := 2 to 2n step 2 do
3 |   sum ← sum + i
4 end
```

Istanza: $n = 4$
(numeri pari: $2, 4, 6, 8$)

i	$i \leq 2n$?	sum
–	–	0
2	Sì	2
4	Sì	6
6	Sì	12
8	Sì	20
10	No	20

Risultato: $sum = 20$

Istruzioni di controllo dei cicli

A volte un ciclo deve essere interrotto
prima della sua fine naturale, oppure
una singola iterazione deve essere saltata.

A questi scopi servono due istruzioni:
break e **continue**.

Istruzione break (1/2)

- L'istruzione **break** esce immediatamente dal ciclo corrente
- L'esecuzione continua con la prima istruzione dopo il ciclo
- Utile quando una condizione di terminazione si verifica prima del completamento naturale del ciclo

Istruzione break (2/2)

```
1 while condizione do
2 |   istruzioni
3 |   if condizione_di_uscita
  |    then
4 |   |   istruzioni
5 |   |   break
6 |   end
7 |   istruzioni
8 end
```

```
1 for variabile := valore_iniziale
   to valore_finale do
2 |   istruzioni
3 |   if condizione_di_uscita
  |    then
4 |   |   istruzioni
5 |   |   break
6 |   end
7 |   istruzioni
8 end
```

Esempio: istruzione break (1/2)

Calcolo della somma dei numeri da 1 a n, fermandosi non appena la somma supera una soglia t.

Con un ciclo while:

```
1 sum := 0
2 i := 1
3 while i ≤ n do
4 |   sum ← sum + i
5 |   if sum > t then
6 |   |   break
7 |   end
8 |   i ← i + 1
9 end
```

Con un ciclo for:

```
1 sum := 0
2 for i := 1 to n do
3 |   sum ← sum + i
4 |   if sum > t then
5 |   |   break
6 |   end
7 end
```

Esempio: istruzione break (2/2)

Esecuzione passo per passo (entrambi i cicli). Istanza: $n = 6$, $t = 10$

i	sum	$sum > t$?
–	0	–
1	1	No
2	3	No
3	6	No
4	10	No
5	15	Sì $\Rightarrow$ **break**

Risultato: $sum = 15$; l'iterazione con $i = 6$ non viene mai eseguita.

Istruzione continue (1/2)

- L'istruzione **continue** salta il resto dell'iterazione corrente
- L'esecuzione prosegue subito con l'iterazione successiva del ciclo (in un ciclo while, con il controllo della condizione)
- Utile quando alcune iterazioni vanno saltate in base a una condizione

Istruzione continue (2/2)

```
1 while condizione do
2 |   istruzioni
3 |   if condizione_di_salto
  |    then
4 |   |   istruzioni
5 |   |   continue
6 |   end
7 |   istruzioni
8 end
```

```
1 for variabile := valore_iniziale
   to valore_finale do
2 |   istruzioni
3 |   if condizione_di_salto
  |    then
4 |   |   istruzioni
5 |   |   continue
6 |   end
7 |   istruzioni
8 end
```

Esempio: istruzione continue (1/3)

Calcolo della somma dei numeri da 1 a n, saltando i multipli di 3.

Con un ciclo while:

```
1  sum := 0
2  i := 1
3  while i ≤ n do
4  |   if i mod 3 = 0 then
5  |   |   i ← i + 1
6  |   |   continue
7  |   end
8  |   sum ← sum + i
9  |   i ← i + 1
10 end
```

Con un ciclo for:

```
1  sum := 0
2  for i := 1 to n do
3  |   if i mod 3 = 0 then
4  |   |   continue
5  |   end
6  |   sum ← sum + i
7  end
```

Esempio: istruzione continue (2/3)

- Nel ciclo while la variabile i deve essere incrementata prima di **continue**
- Altrimenti la condizione $i \bmod 3 = 0$ resterebbe vera per sempre e il ciclo non terminerebbe mai
- Nel ciclo for l'incremento è automatico

Esempio: istruzione continue (3/3)

Esecuzione passo per passo (entrambi i cicli). Istanza: $n = 7$

i	$i \bmod 3 = 0$?	*sum*
–	–	0
1	No	1
2	No	3
3	Sì	3
4	No	7
5	No	12
6	Sì	12
7	No	19

Risultato: $sum = 19 = 1 + 2 + 4 + 5 + 7$

Punti chiave sui costrutti di base

- **if-then** e **if-then-else** scelgono quale blocco eseguire in base a una condizione
- **while** ripete un blocco finché una condizione è vera, controllandola prima di ogni iterazione; **for** ripete un blocco per ogni valore di una variabile del ciclo da un valore iniziale a un valore finale
- **break** esce dal ciclo; **continue** passa all'iterazione successiva
- Eseguire un costrutto passo per passo su una piccola istanza, con una tabella dei valori delle variabili, mostra esattamente che cosa fa

Riepilogo

- **Istruzioni condizionali**: if-then, if-then-else
- **Cicli**: while (anche quando il numero di iterazioni non è noto in anticipo), for (con un passo facoltativo), e l'equivalenza fra un ciclo for e un ciclo while
- **Istruzioni di controllo dei cicli**: break e continue; in un ciclo while la variabile del ciclo va incrementata prima di **continue**

Array

Introduzione: Array

- Molti algoritmi lavorano su una collezione di valori dello stesso tipo, ad esempio una lista di n numeri
- Invece di dichiarare n variabili diverse, si usa un **array**
- Vediamo array con un indice, con due indici e con più indici: come dichiararli, inizializzarli e accedere ai loro elementi

Dichiarazione di un array

- Gli array sono collezioni indicizzate di variabili e devono essere dichiarati prima dell'uso
- L'operatore di dichiarazione è "new array" seguito dall'intervallo degli indici

```
1 nome_array := new array[inizio..fine]
```

Esempio: dichiarazione di un array. Creazione di un array di dimensione n, indicizzato da 1 a n:

```
1 a := new array[1..n]
```

Accesso e inizializzazione di un array

- La notazione $a[i]$ indica la variabile di indice i
- Gli array sono di solito indicizzati a partire da 0 o da 1: nello pseudocodice gli indici partono da 1, quindi un array di n elementi è $a[1..n]$; in molti linguaggi di programmazione, come il C++, gli indici partono da 0
- Ogni variabile dell'array deve essere inizializzata prima di essere letta e usata
- Per inizializzare tutti gli elementi allo stesso valore, si usa un ciclo **for**:

```
1 for i := inizio to fine do
2 |   nome_array[i] ← valore_iniziale
3 end
```

Esempio: assegnamento in un array

Inizializzazione e accesso alle variabili di un array $a[1..n]$ (con $n \geq 5$):

```
// Inizializza a zero tutte le variabili dell'array
1 for i := 1 to n do
2 |   a[i] ← 0
3 end

// Accesso e modifica
4 x := a[5]
5 a[3] ← 42
```

Con $n = 5$ l'array vale $[0, 0, 0, 0, 0]$ dopo il ciclo, $x = 0$, e alla fine l'array vale $[0, 0, 42, 0, 0]$.

Array bidimensionali

Un array può avere anche **due indici**:
un indice di riga e un indice di colonna.

Dichiarazione di un array bidimensionale

- Gli array bidimensionali sono collezioni indicizzate di variabili con due indici e devono essere dichiarati prima dell'uso
- L'operatore di dichiarazione è "new array" seguito dai due intervalli degli indici
- Gli array bidimensionali sono di solito indicizzati a partire da 0 o da 1 (da 1 nello pseudocodice)

```
1 nome_array := new array[inizio_1..fine_1][inizio_2..fine_2]
```

Esempio: dichiarazione di un array bidimensionale. Creazione di un array bidimensionale di dimensione $m \times n$ (indicizzato da 1 a m e da 1 a n):

```
1 b := new array[1..m][1..n]
```

Accesso e inizializzazione di un array bidimensionale

- La notazione $b[i][j]$ indica la variabile nella riga i e nella colonna j
- Ogni variabile dell'array deve essere inizializzata prima di essere letta e usata
- Per inizializzare tutti gli elementi allo stesso valore, si usano cicli **for** annidati:

```
1 for i := inizio_1 to fine_1 do
2 |   for j := inizio_2 to fine_2 do
3 |   |   nome_array[i][j] ← valore_iniziale
4 |   end
5 end
```

Esempio: assegnamento in un array bidimensionale

Inizializzazione e accesso alle variabili di un array bidimensionale $b[1..m][1..n]$ (con $m \geq 2$ e $n \geq 3$):

```
// Inizializza a zero tutte le variabili dell'array
   bidimensionale
1 for i := 1 to m do
2 |   for j := 1 to n do
3 |   |   b[i][j] ← 0
4 |   end
5 end

// Accesso e modifica
6 x := b[2][3]
7 b[1][1] ← 42
```

L'ordine dei cicli annidati

- Nell'esempio precedente il ciclo esterno scorre le righe e, per ogni riga i, il ciclo interno scorre tutte le colonne
- Gli elementi vengono visitati nell'ordine

$$b[1][1], b[1][2], \ldots, b[1][n], b[2][1], \ldots, b[m][n]$$

- per un totale di $m \cdot n$ assegnamenti

Array multidimensionali

Le stesse idee si estendono ad array
con un **numero qualsiasi di indici**.

Dichiarazione di un array multidimensionale

- Gli array multidimensionali sono collezioni indicizzate di variabili con più indici e devono essere dichiarati prima dell'uso
- L'operatore di dichiarazione è "new array" seguito dagli intervalli degli indici per ciascuna dimensione

---

1 nome_array := new
array[inizio$_1$..fine$_1$][inizio$_2$..fine$_2$]$\cdots$[inizio$_k$..fine$_k$]

Esempio: dichiarazione di un array multidimensionale.
Creazione di un array a 3 dimensioni di dimensione $p \times q \times r$ (indicizzato da 1 a p, da 1 a q e da 1 a r):

---

1 t := new array[1..p][1..q][1..r]

Accesso e inizializzazione di un array multidimensionale (1/2)

- La notazione $t[i_1][i_2]\cdots[i_k]$ indica la variabile di indici $i_1, i_2, \ldots, i_k$
- Gli array multidimensionali sono di solito indicizzati a partire da 0 o da 1 (da 1 nello pseudocodice)
- Ogni variabile dell'array deve essere inizializzata prima di essere letta e usata

Accesso e inizializzazione di un array multidimensionale (2/2)

Per inizializzare tutti gli elementi allo stesso valore, si usano cicli **for** annidati (uno per dimensione):

```
1 for i_1 := inizio_1 to fine_1 do
2 |   for i_2 := inizio_2 to fine_2 do
  |   |   ⋮
3 |   |   for i_k := inizio_k to fine_k do
4 |   |   |   nome_array[i_1][i_2]···[i_k] ← valore_iniziale
5 |   |   end
6 |   end
7 end
8
```

Esempio: assegnamento in un array multidimensionale

Inizializzazione e accesso a un array a 3 dimensioni $t[1..p][1..q][1..r]$ (con $p \geq 2$, $q \geq 2$ e $r \geq 3$):

```
   // Inizializza a zero tutte le variabili dell'array
      a 3 dimensioni
1  for i := 1 to p do
2  |   for j := 1 to q do
3  |   |   for k := 1 to r do
4  |   |   |   t[i][j][k] ← 0
5  |   |   end
6  |   end
7  end

   // Accesso e modifica
8  x := t[1][2][3]
9  t[2][1][3] ← 42
```

Riepilogo

- Un array si dichiara con **new array** e i suoi intervalli di indici: $a[1..n]$, $b[1..m][1..n]$, $t[1..p][1..q][1..r]$
- Nello pseudocodice gli indici partono da 1; in molti linguaggi di programmazione, come il C++, da 0
- Ogni elemento va inizializzato prima dell'uso: un ciclo **for** per dimensione, annidati

Funzioni

Introduzione: Funzioni

- Gli esempi visti finora sono frammenti di pseudocodice che lavorano su variabili supposte esistenti
- Una **funzione** trasforma un frammento di questo tipo in un algoritmo completo, con un nome, un input e un output
- D'ora in poi ogni algoritmo del corso è scritto come una funzione: ne vediamo la sintassi e cinque algoritmi completi sugli array

Funzioni

- Una **funzione** è un blocco di codice dotato di nome e riutilizzabile che svolge un compito specifico
- Una funzione riceve dati di **input** attraverso i suoi **parametri** e produce dati di **output**
- L'istruzione **return** termina la funzione e ne specifica i risultati
- Le funzioni si usano di solito per realizzare **algoritmi** o per eseguire calcoli specifici che possono essere riutilizzati in parti diverse di un programma

Sintassi di una funzione

Input : descrizione dei parametri di input
Output: descrizione dell'output / del valore restituito

```
1 function FunctionName(parametro_1, parametro_2, ..., parametro_k)
2 |   istruzioni
3 |   return risultato
4 end
```

Algoritmi come funzioni

- D'ora in poi ogni algoritmo è scritto come una funzione, dentro un riquadro rosso
- L'input indica le **precondizioni** sui dati (ad esempio, che un array abbia almeno un elemento)
- La funzione è garantita funzionare solo quando le precondizioni valgono

Esempio: valore massimo in un array (1/4)

- Una funzione che restituisce il valore massimo di un array
- La variabile *max* parte dal primo elemento e viene aggiornata ogni volta che si trova un elemento più grande

Esempio: valore massimo in un array (2/4)

Algorithm: FindMaximum

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output: Il valore massimo nell'array a

```
1 function FindMaximum(a, n)
2 |   max := a[1]
3 |   for i := 2 to n do
4 |   |   if a[i] > max then
5 |   |   |   max ← a[i]
6 |   |   end
7 |   end
8 |   return max
9 end
```

Esegui in Colab: Python · C++ · Java

Esempio: valore massimo in un array (3/4)

Esecuzione passo per passo. Istanza 1:
$a = [9, 12, 3, 16, 19, 11, 12]$, $n = 7$

i	$a[i]$	$a[i] > max$?	max
–	–	–	9
2	12	Sì	12
3	3	No	12
4	16	Sì	16
5	19	Sì	19
6	11	No	19
7	12	No	19

Restituisce: $max = 19$

Esempio: valore massimo in un array (4/4)

Esecuzione passo per passo. Istanza 2: $a = [3, 7, 2, 9, 1]$, $n = 5$

i	$a[i]$	$a[i] > max$?	max
–	–	–	3
2	7	Sì	7
3	2	No	7
4	9	Sì	9
5	1	No	9

Restituisce: $max = 9$

Esempio: conteggio degli elementi positivi di un array (1/3)

Una funzione che conta il numero di elementi positivi in un array:

Algorithm: CountPositive

Input : Un array $a[1..n]$ di n numeri interi
Output : Il numero di elementi positivi nell'array a

```
1 function CountPositive(a, n)
2     count := 0
3     for i := 1 to n do
4         if a[i] > 0 then
5             count ← count + 1
6         end
7     end
8     return count
9 end
```

Esegui in Colab: Python · C++ · Java

Esempio: conteggio degli elementi positivi di un array (2/3)

Esecuzione passo per passo. Istanza 1: $a = [-2, 5, -1, 3, 0]$, $n = 5$

i	$a[i]$	$a[i] > 0$?	*count*
–	–	–	0
1	-2	No	0
2	5	Sì	1
3	-1	No	1
4	3	Sì	2
5	0	No	2

Restituisce: $count = 2$

Esempio: conteggio degli elementi positivi di un array (3/3)

Esecuzione passo per passo. Istanza 2: $a = [4, -3, 7, 2]$, $n = 4$

i	$a[i]$	$a[i] > 0$?	*count*
–	–	–	0
1	4	Sì	1
2	-3	No	1
3	7	Sì	2
4	2	Sì	3

Restituisce: $count = 3$

Esempio: verifica se un array è ordinato (1/3)

- Una funzione che verifica se un array è ordinato in ordine non decrescente
- L'istruzione **return** dentro il ciclo termina la funzione non appena si trovano due elementi consecutivi nell'ordine sbagliato

Esempio: verifica se un array è ordinato (2/3)

Algorithm: IsSorted

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output: **true** se i valori dell'array a sono ordinati in ordine non decrescente e **false** altrimenti

```
1 function IsSorted(a, n)
2     for i := 1 to n − 1 do
3         if a[i] > a[i + 1] then
4             return false
5         end
6     end
7     return true
8 end
```

Esegui in Colab: Python · C++ · Java

Esempio: verifica se un array è ordinato (3/3)

Esecuzione passo per passo.

Istanza 1: $a = [2, 5, 7, 9]$, $n = 4$

i	$a[i]$	$a[i+1]$	$a[i] > a[i+1]$?
1	2	5	No
2	5	7	No
3	7	9	No

Restituisce: true

Istanza 2: $a = [3, 1, 4, 2]$, $n = 4$

i	$a[i]$	$a[i+1]$	$a[i] > a[i+1]$?
1	3	1	Sì

Restituisce: false, dopo un solo confronto.

Esempio: prodotto scalare di due array (1/3)

Una funzione che calcola il prodotto scalare $\sum_{i=1}^{n} a[i] \cdot b[i]$ di due array:

Algorithm: ScalarProduct

Input : Due array $a[1..n]$ e $b[1..n]$ di dimensione $n \geq 1$
Output: Il prodotto scalare dei due array

```
1 function ScalarProduct(a, b, n)
2 |   prod := a[1] · b[1]
3 |   for i := 2 to n do
4 |   |   prod ← prod + a[i] · b[i]
5 |   end
6 |   return prod
7 end
```

Esegui in Colab: Python · C++ · Java

Esempio: prodotto scalare di due array (2/3)

Esecuzione passo per passo. Istanza 1: $a = [1, 2, 3]$, $b = [4, 5, 6]$, $n = 3$

i	$a[i]$	$b[i]$	$a[i] \cdot b[i]$	$prod$
–	–	–	–	$1 \cdot 4 = 4$
2	2	5	10	$4 + 10 = 14$
3	3	6	18	$14 + 18 = 32$

Restituisce: $prod = 32$

Esempio: prodotto scalare di due array (3/3)

Esecuzione passo per passo. Istanza 2: $a = [2, 0, -1]$, $b = [3, 4, 5]$, $n = 3$

i	$a[i]$	$b[i]$	$a[i] \cdot b[i]$	$prod$
–	–	–	–	$2 \cdot 3 = 6$
2	0	4	0	$6 + 0 = 6$
3	-1	5	-5	$6 + (-5) = 1$

Restituisce: $prod = 1$

Esempio: inversione di un array (1/4)

- Una funzione che inverte l'ordine degli elementi di un array
- L'elemento $a[i]$ viene scambiato con il suo simmetrico $a[n - i + 1]$, con una variabile temporanea, come nello scambio dei valori di due variabili
- Bastano $\lfloor n/2 \rfloor$ scambi: con n dispari, l'elemento centrale resta al suo posto

Esempio: inversione di un array (2/4)

Algorithm: ReverseArray

Input : Un array $a[1..n]$ di n numeri
Output: Lo stesso array a con gli elementi in ordine inverso

```
1 function ReverseArray(a, n)
2 |   for i := 1 to ⌊n/2⌋ do
3 |   |   tmp := a[i]
4 |   |   a[i] ← a[n − i + 1]
5 |   |   a[n − i + 1] ← tmp
6 |   end
7 |   return a
8 end
```

Esegui in Colab: Python · C++ · Java

Esempio: inversione di un array (3/4)

Esecuzione passo per passo. Istanza 1: $a = [1, 2, 3, 4, 5]$, $n = 5$, $\lfloor 5/2 \rfloor = 2$ iterazioni

i	scambio	array a (dopo lo scambio)
–	–	$[1, 2, 3, 4, 5]$
1	$a[1] \leftrightarrow a[5]$	$[5, 2, 3, 4, 1]$
2	$a[2] \leftrightarrow a[4]$	$[5, 4, 3, 2, 1]$

Restituisce: $a = [5, 4, 3, 2, 1]$

Esempio: inversione di un array (4/4)

Esecuzione passo per passo. Istanza 2: $a = [10, 20, 30, 40]$, $n = 4$, $\lfloor 4/2 \rfloor = 2$ iterazioni

i	scambio	array a (dopo lo scambio)
–	–	$[10, 20, 30, 40]$
1	$a[1] \leftrightarrow a[4]$	$[40, 20, 30, 10]$
2	$a[2] \leftrightarrow a[3]$	$[40, 30, 20, 10]$

Restituisce: $a = [40, 30, 20, 10]$

Riepilogo

- Una **funzione** ha un nome, dei parametri, un input con le sue precondizioni, un output, e un'istruzione **return** che la termina
- Cinque algoritmi completi sugli array: FindMaximum, CountPositive, IsSorted, ScalarProduct, ReverseArray
- Un **return** dentro un ciclo termina la funzione in anticipo, come in IsSorted

Insiemi

Introduzione: Insiemi

- Gli array memorizzano valori in posizioni
- Quando conta soltanto l'appartenenza dei valori, e non la loro posizione o le loro ripetizioni, si usa un **insieme**
- Vediamo come dichiarare un insieme, i suoi operatori principali e come scorrerne gli elementi

Insiemi

- Un **insieme** (*set*) è una collezione non ordinata di elementi *distinti*
- Gli insiemi devono essere dichiarati prima dell'uso con l'operatore **new set**; non occorre specificare alcuna dimensione; appena creato, un insieme è vuoto

```
1 nome_insieme := new set
```

Operatori sugli insiemi (1/2)

I principali operatori sugli insiemi sono:

- **add**: inserisce un elemento nell'insieme (ignorato se già presente)

  add x **to** nome_insieme

- **remove**: elimina un elemento dall'insieme (ignorato se non presente)

  remove x **from** nome_insieme

- **in**: test di appartenenza, che vale **true** se x appartiene all'insieme

  x **in** nome_insieme

Operatori sugli insiemi (2/2)

- **size**: cardinalità (numero di elementi attualmente nell'insieme)

size(nome_insieme)

- **foreach**: ciclo sugli elementi dell'insieme; per convenzione, gli elementi vengono visitati nell'ordine di inserimento

```
1 foreach x in nome_insieme do
  |  // elabora x
2 end
```

- È possibile scorrere gli elementi di un insieme in un ordine specifico indicando un criterio di ordinamento:

```
1 foreach x in nome_insieme sorted by criterio do
  |  // elabora x
2 end
```

Esempio: dichiarazione e uso di un insieme (1/4)

```
   // Dichiara un insieme vuoto
1  s := new set

   // Riempie l'insieme con i numeri pari da 1 a n
2  for i := 1 to n do
3  |   if i mod 2 = 0 then
4  |   |   add i to s
5  |   end
6  end

   // Test di appartenenza
7  if 4 in s then
8  |   print(4 appartiene a s)
9  end
```

Esempio: dichiarazione e uso di un insieme (2/4)

```
   // Rimuove un elemento
 9 remove 4 from s

   // Scorre e stampa tutti gli elementi rimasti
10 foreach x in s do
11 |   print(x)
12 end

   // Stampa la cardinalità
13 print(size(s))
```

Esempio: dichiarazione e uso di un insieme (3/4)

```
   // Stampa gli elementi in ordine non crescente
13 foreach x in s sorted by valore non crescente do
14 |   print(x)
15 end

   // Stampa gli elementi in ordine non decrescente
16 foreach x in s sorted by valore non decrescente do
17 |   print(x)
18 end
```

Esempio: dichiarazione e uso di un insieme (4/4)

Esecuzione passo per passo. Istanza: $n = 8$

- Passo 1: il ciclo for aggiunge $2, 4, 6, 8$, quindi $s = \{2, 4, 6, 8\}$
- Passo 2: 4 appartiene a s: l'output è `4 appartiene a s`
- Passo 3: dopo la rimozione, $s = \{2, 6, 8\}$
- Passo 4: il primo **foreach** stampa $2, 6, 8$ (ordine di inserimento), poi viene stampata la cardinalità 3
- Passo 5: gli ultimi due cicli stampano $8, 6, 2$ e $2, 6, 8$

Riepilogo

- Un **insieme** è una collezione non ordinata di elementi distinti, dichiarata vuota con **new set**
- Operatori: **add**, **remove**, **in**, **size**
- **foreach** visita gli elementi nell'ordine di inserimento, o nell'ordine dato da **sorted by**

Convenzioni di stile

Introduzione: Convenzioni di stile

- Concludiamo con un riepilogo della notazione usata nel corso
- La confrontiamo con altre scelte comuni, che si possono trovare nei libri e in altri corsi

Convenzioni diverse

Autori diversi possono usare convenzioni leggermente diverse:

- Dichiarazione: $:=$ (usata in questo corso) oppure $=$ (alcuni autori usano lo stesso simbolo sia per la dichiarazione sia per l'assegnamento)
- Assegnamento: $\leftarrow$ (usato in questo corso) oppure $=$
- Indicizzazione di un array: $a[i]$ (usata in questo corso) oppure a_i
- Indicizzazione di un array bidimensionale: $b[i][j]$ (usata in questo corso) oppure b_{ij}
- Commenti: // commento, # commento, oppure (* commento *)

L'importante è essere coerenti e chiari!

Riepilogo

- In questo corso: dichiarazione $:=$, assegnamento $\leftarrow$, indicizzazione $a[i]$ e $b[i][j]$
- Altri autori possono scrivere $=$, a_i, b_{ij}, e altri simboli per i commenti
- Qualunque sia la notazione, bisogna essere **coerenti e chiari**

Riepilogo: punti chiave del capitolo

- Le variabili si dichiarano con $:=$ e si modificano con $\leftarrow$
- Le istruzioni condizionali e i cicli (**if**, **while**, **for**, con **break** e **continue**) controllano l'ordine di esecuzione
- Gli array $a[1..n]$, $b[1..m][1..n]$, ... sono indicizzati da 1 nello pseudocodice; gli insiemi memorizzano elementi distinti senza posizioni
- Un algoritmo si scrive come una **funzione** con il suo input (comprese le precondizioni), il suo output e un'istruzione **return**